Unidad 1 / Escenario 1

Lectura fundamental

Conceptos iniciales en lenguajes de programación

Contenido

1 Vocabulario básico de programación 2

2 Clasificación de lenguajes de programación 3

3 Java 4

4 Tipos de datos y sus operaciones 5

5 Problemas propuestos 25

Palabras clave: Lenguajes de programación, imperativo, Java

1. Vocabulario básico de programación

En esta sección se presenta el vocabulario básico que se necesita para iniciar el aprendizaje de la programación, su repaso e interiorización. Para cada término se intenta dar una definición sencilla y clara con el fin de facilitar el inicio de la práctica de programación, la cual es el eje principal del módulo.

1. Programa (de computador): es básicamente el conjunto de instrucciones que le dicen qué hacer al computador.
2. IDE *(Integrated Development Environment)* o entorno integrado de desarrollo: es un programa que combina varias herramientas que permiten crear programas; normalmente, tiene al menos un editor de texto, ya sea integrado o vinculado, un compilador o intérprete de un lenguaje de programación.
3. Lenguaje de programación: conjunto de palabras e instrucciones básicas para crear un programa de computador (*software*).
4. Depurar (programación): proceso de búsqueda y detección de errores en programas de computador.
5. Compilador: programa de computador que toma un escrito *(script)* o conjunto de ellos hechos en algún lenguaje de programación, para producir un archivo ejecutable: un programa (o que se pueda interpretar por otro programa).
6. Compilar: es el proceso de tomar uno o más archivos escritos en un lenguaje de programación y producir un archivo ejecutable en un computador.
7. Ejecutar (un programa).
8. Intérprete (programación): es un programa que toma instrucción por instrucción de otro programa o secuencia de instrucciones y las ejecuta paso a paso.
9. Entrada estándar de datos: todo lenguaje de programación de propósito general tiene una o más opciones de leer datos de manera estándar; esta es su herramienta de recolección de datos más sencilla. En el caso de Java hay varias, siendo la clase *Scanner* la más fácil de usar.
10. Salida estándar de datos: al igual que con la entrada estándar, los lenguajes de programación de propósito general tienen al menos una manera de producir datos a la salida estándar del lenguaje. En el caso de Java, la más sencilla es la dada por los métodos *print* y *println* de la clase *System*, en la instancia *out*. Es decir, son: *System.out.print() y System.out.println().*

2. Clasificación de lenguajes de programación

Los lenguajes de programación se pueden clasificar de diferentes maneras. Una consiste en los paradigmas de programación, que incluyen los orientados a objetos (como Java), los funcionales y los lógicos, entre otros. También se pueden clasificar por generaciones o niveles, siendo los primeros, o los de bajo nivel, los más antiguos y cercanos a lo que entiende una máquina (más relacionados con el *hardware*); los de últimas generaciones son los más cercanos a lo que las personas comprenden como instrucciones. Java es un ejemplo de lenguaje de alto nivel de tercera generación. Otra forma no tan común de clasificar lenguajes se refiere a si son tipados o no, o si son compilados o interpretados, o ambos. El lenguaje Java, es un lenguaje de programación de alto nivel, compilado e interpretado, tipado y orientado a objetos. Algunas implementaciones de Java usan compilación *Just-in-time*, la cual es un proceso en el que se compila el código mientras se ejecuta. Para conocer más detalles sobre paradigmas y tipos de lenguajes de programación, ver Watt (2004); Louden y Lambert (2011); Gabbrielli y Martini (2010); Van Roy y Haridi (2004) y Sebesta (2012).

3. Java

Aunque todos los ejemplos aquí mostrados están en Java, los conceptos expuestos son transversales a todos los lenguajes de programación y muchas de las operaciones/herramientas estándar de Java tienen equivalente en otros lenguajes. Java goza de una gran historia y reputación por su uso en el ámbito empresarial y académico, razones por las cuales es una buena selección como primer lenguaje de programación. Para más detalles sobre Java, revisar Putano (2017) y Blair (2017); por otro lado, para profundizar en más aspectos se recomiendan lecturas de libros especializados en Java como lenguaje; ver Sierra y Bates (2005) y Deitel y Deitel (1999).

Tabla 1: tipos de datos nativos en Java

Tipo Primitivo	Tamaño en bits	Tipo	Valor minimo	Valor mínimo en binario	Valor Máximo	Valor máximo en binario
byte	8 bits	Entero	-128	10000000	127	1111111
short	16 bits	Entero	-32768	10000000 00000000	32767	11111111 1111111
int	32 bits	Entero	-2147483648	10000000 00000000 00000000 00000000	2147483647	11111111 11111111 11111111 1111111
long	64 bits	Entero	-9223372036854775808	10000000 00000000 00000000 00000000 00000000 00000000 00000000 0000000	9223372036854775807	11111111 11111111 11111111 11111111 11111111 11111111 11111111 1111111
float	32 bits	Real	1.40E-45		3.40E+38	
double	64 bits	Real	5.00E-324		1.80E+308	
char	16 bits	Caracter				
boolean	1 bit*	Booleano				

Fuente: elaboración propia

4. Tipos de datos y sus operaciones

Todo lenguaje de programación tipado, como Java, tiene una clasificación de tipos de datos. Los más comunes son los numéricos, los booleanos, los de carácter y los de cadena. La gran mayoría de lenguajes de programación tienen estos tipos de datos disponibles de manera nativa (también conocidos como primitivos). Los tipos de datos nativos son aquellos que tienen los lenguajes como unidades básicas de almacenamiento. En todos los lenguajes de programación es absolutamente necesario tener presente el conjunto de valores que se puede almacenar en ellos; esto se puede resumir conociendo el tipo de dato, su valor máximo, su valor mínimo y, en algunos casos, su representación en memoria. La Tabla 1 resume los tipos de datos nativos en Java, los cuales se pueden dividir en:

1. Tipo entero: *byte, short, int, long*. Sirven para almacenar valores numéricos enteros. La selección del dato a usar depende del contexto y de la magnitud de los valores que se van a almacenar, es decir, de la capacidad de almacenamiento de estos datos.
2. Tipo carácter: *char*. Almacenan fundamentalmente caracteres o símbolos. Internamente no son diferentes a un tipo entero, en cuanto que almacenan un valor de entero que corresponde a un símbolo o carácter en el sistema de codificación de caracteres que se tenga; uno de los más usados es UNICODE.
3. Tipo booleano: *boolean*. Sirven para almacenar un conjunto de solo dos valores, a saber: falso y verdadero; los valores válidos son, entonces, *false* y *true*.
4. Tipo número real: *float, double*. Sirven para almacenar valores numéricos reales, es decir, con parte decimal, tales como PI, que es aproximadamente 3.1416. Dependiendo de la precisión que se desea tener, se usa uno de los dos. El *double* es representado usando 64 bits, con 1 bit de signo, 11 bits para el exponente y 52 bits para la parte significativa del número. El *float* es representado usando 32 bits, con 1 bit de signo, 8 bits para el exponente y 23 bits para la parte significativa del número. Ver Oracle (1995) para más detalles.

4.1. Selección de los tipos de datos

Uno de los primeros pasos después de que se tiene la solución a un problema o la aproximación a una solución de programación y desarrollo consiste en seleccionar los tipos de datos a usar para almacenar cada porción de la información que se va a procesar. Por esto, los primeros conceptos que se deben tener en mente son los tipos de datos disponibles en cada lenguaje. De esta manera, cuando un programador se enfrenta a un problema o situación en la que se deba almacenar temporal o permanentemente alguna información, puede decidir qué tipo de dato usar. Hay algunos lenguajes de programación llamados no tipados, en los cuales se deja esta tarea al compilador o intérprete del lenguaje, como es el caso de Python. De cierta manera, los lenguajes de programación se pueden clasificar respecto a si son tipados o no, y se tiene un espectro de posibilidades, como sucede en muchas clasificaciones. Sin embargo, la discusión sobre este tema se escapa del alcance del presente módulo.

Para propósitos de este módulo, se considera Java como un lenguaje tipado puesto que cada valor a almacenar se debe declarar como perteneciente a un tipo, ya sea nativo o definido por el estándar de Java o por el programador. Para más información sobre tipado en lenguaje de programación, consultar Ovid (2010); para discusiones en grupos de interés sobre si algún lenguaje específico es tipado o no, o qué tan tipado es, remitirse a StackOverflow (2018) y StackExchange (2018).

Como un ejemplo para seleccionar el tipo de dato, considere las siguientes situaciones. Cuando se requiere almacenar la información sobre si una persona ya realizó una acción o no, se debe escoger un tipo de dato booleano, ya que este puede almacenar uno de dos valores: *true* o *false*. Si se desea almacenar la cantidad de veces que una persona se ha registrado a un sistema, sirve un tipo de dato entero, siendo el más usado el tipo *int*, aunque cabe analizar si se puede o no usar un tipo de dato entero que ocupe menos espacio. En el caso que estemos calculando y almacenando magnitudes físicas, como pesos, alturas, volúmenes, áreas, etc., casi con toda seguridad conviene usar tipos de datos como *float* o *double*. Es importante aclarar que algunos lenguajes tienen tipos de datos más complejos y sofisticados que vale la pena tenerlos en cuenta para posibles aplicaciones en el desarrollo profesional de los programadores. Una vez dominados los conceptos de tipos de datos básicos (o nativos) en un lenguaje de programación, con un poco de lectura y práctica adicional se podrán manejar tipos de datos más sofisticados y usarlos efectivamente en proyectos de desarrollo de *software*.

4.2. Nombramiento de variables

Después de seleccionar el tipo de dato conveniente para almacenar cierta información, el siguiente paso es ponerle un nombre a ese espacio de memoria. Todo programador debe manejar este proceso, que se conoce como nombramiento de variables, de manera casi automática; con el tiempo y la práctica suficientes, esto será una realidad para quienes cursan este módulo. Las restricciones para nombramiento de variables en los lenguajes de programación generalmente son pocas, siendo las más comunes las siguientes: no usar palabras reservadas [para Java, Oracle (1995g)]; se debe empezar por letra o guion al piso y después de este primer símbolo del nombre, solo deben ir letras y números (sin espacios, ni tabulaciones, ni saltos de línea).

En Java esas recomendaciones funcionan, aunque vale la pena anotar que es posible hacer declaraciones como *int* \u0050; e *int* $a;, pero se omitirán sus usos para fomentar las buenas prácticas de programación en cuanto a legibilidad de código. También se intentará desincentivar el empleo de guiones al piso en nombramiento de clases, métodos y variables, aunque en ciertos casos estos pueden ayudar a la legibilidad.

Algunos lenguajes son sensibles a las mayúsculas y a las minúsculas (como Java, C++ y otros), en cuyo caso las variables nombradas *count*, *Count* y *COUNT* serán consideradas diferentes y se debe hacer una declaración para cada una. Nombrar tres variables así en un mismo ámbito de un programa probablemente es una muy mala decisión en cuanto a buenas prácticas de programación, puesto que no favorece para nada la legibilidad del código. Algunos otros lenguajes no son sensibles a mayúsculas-minúsculas, como SQL, Pascal, Delphi y Basic.

Para propósitos de programas cortos y pequeños, no está mal nombrar variables con una sola letra, pero a medida que los programas crecen en extensión y complejidad conviene nombrarlas usando nombres descriptivos. Estos nombres deben respetar el estándar *lowerCamelCase*, que básicamente quiere decir que deben empezar en minúscula, y que en cada cambio de palabra debe haber una mayúscula; por ejemplo: *peopleCounter, lastName*, *moneyAccumulator.* Vale la pena aclarar que las clases en Java se nombran respetando el estándar *UpperCamelCase*, que es muy similar al *lowerCamelCase*, con la diferencia de que se debe empezar con mayúscula. De esta manera, deben nombrarse todas las clases hechas en este módulo; por ejemplo: *Main, MainProgram, LoopExample*. Adicional a estas convenciones, en Java existe una para nombrar constantes: con mayúscula sostenida, y en donde haya separación de palabras debe ir un guion al piso; por ejemplo: *MAX_VALUE*, *TOTAL_SPACE.* Las constantes en Java son valores almacenados que se asume (y el compilador se asegura de ello) que no van a cambiar durante toda la ejecución del programa; se declaran con ayuda de la palabra reservada final.

Por último, pero no menos importante, todo equipo de programación o empresa de desarrollo de *software* debe fijar estándares en el nombramiento de variables y, en general, en el lenguaje escogido para nombrar artefactos y documentar dentro del código. Para favorecer tanto la práctica del inglés como la posibilidad de aprender a programar de una manera que se pueda compartir e internacionalizar mejor, se invita tanto a estudiantes como a los tutores de este módulo a usar ese idioma para el nombramiento de variables, métodos, clases y demás artefactos, como para la documentación del código. Aparte de la obvia ventaja de practicar el inglés, la razón más importante para usarlo en todos los artefactos es que, en caso de dudas o preguntas, se puede recurrir a las grandes comunidades internacionales como apoyo [StackOverflow (2018); Quora (2018); TopCoder (2018); StackExchange (2018)]. Para ello, solo hay que copiar y pegar el código en foros o plataformas adecuadas y millares de programadores y estudiantes alrededor del mundo pueden ayudarle; mientras que si se usa el español, solo una fracción de posibles colaboradores lo puede entender.

4.3. Declaración de variables

En todo lenguaje de programación de alto nivel hay numerosas maneras de declarar variables. Las variables son el nombre genérico que se le da en programación a los espacios en la memoria del computador que sirven a los programadores para almacenar información. Teniendo en cuenta los tipos de datos básicos, en el siguiente programa se muestran las declaraciones más comunes de tipos de datos nativos, la forma en que se pueden leer de la entrada estándar y cómo se ven impresos en la salida estándar del lenguaje. Para más detalles, se recomienda siempre revisar la documentación oficial de cada lenguaje y practicar resolviendo la mayor cantidad posible de ejercicios relacionados. Se aprovecha este programa para mostrar una buena opción de cómo se debe documentar el código escrito usando comentarios en este. Los comentarios son parte del escrito de código que no se ejecutan; pueden estar en una sola línea después de // o en múltiples líneas encerrados entre /* y */. Para conocer más detalladamente cómo comentar un código en cualquier lenguaje siempre se puede acudir a la documentación oficial, que para el caso de Java es Oracle (1999a).

```
import
java.util.Scanner; public
class Main {
    public static void main(String args[] ){
        //Here we declare a variable called sn, wich is an instance of Scanner
        //Is just for read the data.
        Scanner sn = new Scanner(System.in);

        //The next 9 lines are valid declarations of native data type variables.
        byte b;
        short s;
        int     i;
        long l;
        float f;
        double d;
        char c;
        boolean     bn;

        //The next line is a example of how to declare a String variable.
        String str;

        System.out.print("Type a byte value: ");
        b = sn.nextByte();
        System.out.println("The byte we just read is: "+b);

        System.out.print("Type a short value: ");
        s = sn.nextShort();
        System.out.println("The short we just read is: "+s);

        System.out.print("Type a int value: ");
        i = sn.nextInt();
        System.out.println("The int we just read is: "+i);

        System.out.print("Type a long value: ");
        l= sn.nextLong();
        System.out.println("The long we just read is: "+l);
    }
}
```

Código 1: ejemplo de declaración y lectura de datos con diferentes tipos 1/2

```
        System.out.print("Type a float value (a number like 4.2): ");
        f = sn.nextFloat();
        System.out.println("The float we just read is: "+f);

        System.out.print("Type a double value (a number like 4.32332): "); d =
        sn.nextDouble();
        System.out.println("The double we just read is: "+d);

        //The next two lines does not work
        //c = sn.nextChar();
        //System.out.println("The  we just read is: "+);

        //There is no support in Java to read a char.
        //So we read a String and extract the first char
        System.out.print("Type a char value (a single letter): ");
        c = sn.next().charAt(0);
        System.out.println("The  we just read is: "+c);

        System.out.print("Type a boolean value (true or false): ");
        bn = sn.nextBoolean();
        System.out.println("The boolean we just read is: "+bn);

        System.out.print("Type a String value (without spaces or tabs): ");
        str = sn.next();
        System.out.println("The String we just read is: "+str);

        sn.close();
    }
}
```

Código 2: ejemplo de declaración y lectura de datos con diferentes tipos 2/2

4.4. Otros tipos de datos

Hay otros tipos de datos más complejos o sofisticados, o diseñados para aplicaciones más específicas que, aunque se escapan del alcance de este módulo, vale la pena considerarlos en el aprendizaje de programación y en un futuro desarrollo como profesionales en computación. Tener presente la existencia de estos tipos de datos es importante, ya que la labor del programador exige un continuo aprendizaje de nuevos conceptos, lenguajes, API y tecnologías en general. Entre estos tipos de datos exóticos se pueden encontrar las tuplas en lenguajes como Standard ML, Python, Scala, Swift, C++ y Elixir; también en las listas de lenguajes como *Common Lisp, Scheme* y *Haskell*. De igual manera, algunos lenguajes tienen la posibilidad de trabajar con números complejos en sus librerías estándar (como C99, Fortran, Common Lisp, Python, D, Go) o números racionales (como en Common Lisp y Haskell); los arreglos asociativos son otro tipo de datos estándar en *Perl, Python, Ruby, Javascript, Lua, D, Go*. Finalmente, una de las últimas adiciones en Java8 son las funciones de primera clase (*First-class function*), estándar que también está presente en prácticamente todos los lenguajes funcionales, como *Javascript, Lua, D, Go* y en lanzamientos recientes de C++, C# y Perl.

Otro dato frecuentemente usado en sistemas computacionales, y para el cual algunos lenguajes como Java tienen soporte, es el tipo de dato fecha. Las fechas en Java se pueden manejar de varias maneras; la clase *GregorianCalendar* tiene bastantes utilidades, como se puede ver en la documentación de Oracle (1999c).

4.5. Operadores y expresiones

Las operaciones más comunes en los lenguajes de programación son las aritméticas, booleanas y de cadenas. En todas ellas se debe respetar la precedencia de operaciones de acuerdo con los paréntesis que haya a lugar. En caso de que no estén, se sigue el orden de operaciones dado por la prioridad en los operadores, como lo indica la Tabla 2. Aunque cualquier programador en su ejercicio profesional deber conocer estos datos, es más importante entender cómo usarlos y cómo salir de dudas en caso de tenerlos en algún lenguaje o a la hora de escoger cómo hacer alguna expresión u operación. Para ello, el mejor consejo es practicar lo más que se pueda, ya sea con ejercicios dados en el módulo, o con ejercicios y problemas que se pueden encontrar en recursos sugeridos por el tutor. La siguiente tabla resume los operadores y su precedencia:

Tabla 2. Operadores en Java

Operators	Precedence
postfix	expr++ expr–
unary	++expr –expr +expr -expr ~!
multiplicative	* / %
additive	+ -
shift	<< >> >>>
relational	<><= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	|
logical AND	&&
logical OR	||
ternary	? :
assignment	= += -= *= /= %= &= ^= —= <<= >>= >>>=

Fuente: elaboración propia

4.6. Operaciones aritméticas

En casi todos los lenguajes de programación de propósito general es posible formular expresiones aritméticas. Estas tienen, fundamentalmente, dos tipos de resultados: enteros {..., −3, −2, −1, 0, 1, 2, 3, ...} y reales (como, por ejemplo, 0.0, 10e − 3, 3.1416). Para más información en valores posibles de literales, revisar la documentación de cada lenguaje [Oracle (1999b) en el caso de Java]. En Java, y en muchos otros lenguajes de programación, los posibles resultados de una operación matemática dependen de procesos que realiza el compilador y/o intérprete del lenguaje; estos procesos se conocen como conversiones, promociones y castings. Para conocer en más detalle este tema, se recomienda que revise la documentación; para Java ver Oracle (1995e).

La recomendación es practicar y analizar con detenimiento cada expresión diseñada para encontrar posibles errores de análisis, matemáticos o de conceptos. Vale la pena anotar que la operación asignación se hace usando el operador igual = y se realiza de derecha a izquierda; en la parte izquierda debe haber una

variable y en la derecha una expresión que puede ser una constante o una operación correctamente escrita que dé como resultado un valor que se pueda asignar a la variable de la izquierda. Las operaciones se realizan según la prioridad dada por la Tabla 2.

4.6.1. Operaciones en tipos de datos reales

Las operaciones que involucran tipos de datos numéricos en los que alguno de sus literales, variables, o en general alguno de sus valores es real, resultarán en un valor real, es decir, un *double* o un *float*. En el Código 3 se observan varias operaciones y asignaciones posibles; aunque en los ejemplos se usa solo *doubles* e *ints*, el análisis se puede extender a *float* y a otros tipos de datos nativos enteros. Es importante notar en este y en futuros ejemplos de código, que en muchas ocasiones conviene dejar unos espacios de más en las expresiones y declaraciones con el objetivo de favorecer la legibilidad de los programas; estos espacios pueden separar sumandos, declaraciones u operaciones complejas como se ve en la asignación de la variable *doubleJ*.

```
public class DoubleExample{
    public static void main( String args[]){
        int intA = 4;
        int intB = 3;
        double doubleA = 4;
        double doubleB = 3;
        double doubleC = 4/3.0;
        double doubleD = 4.0/3;
        double doubleE = 4.0/3.0;
        double doubleF = intA/intB;
        double doubleG = intA/(double)intB;
        double doubleH = (double)intA/intB;
        double doubleI = (double)intA/(double)intB;
        double doubleJ = 4/5 + 3*intB/intA - intA/intB + (intA*(intB/intA));

        System.out.println("doubleA      = "+doubleA);
        System.out.println("doubleB      = "+doubleB);
        System.out.println("doubleC      = "+doubleC);
        System.out.println("doubleD      = "+doubleD);
        System.out.println("doubleE      = "+doubleE);
        System.out.println("doubleF      = "+doubleF);
        System.out.println("doubleG      = "+doubleG);
        System.out.println("doubleH      = "+doubleH);
        System.out.println("doubleI      = "+doubleI);
        System.out.println("doubleJ      = "+doubleJ);
    }
}
```

Código 3: ejemplos de operaciones entre números reales

4.6.2. Operaciones en tipos de datos enteros

Los operadores disponibles para trabajar con números enteros son los mismos que se usan con números reales, más unos especiales de uso muy común para tipos de datos enteros: cociente y residuo de división. El operador división /, cuando es aplicado entre dos valores o expresiones enteras, da como resultado el cociente de la división; por ello, en el Código 3 el valor de la variable *doubleF* es 1, que, al ser asignado a una variable tipo *double*, se imprime como 1.0, pero como en su cálculo solo hay valores enteros, este también lo es.

Adicional a la obtención del cociente por medio de la división entre enteros, estos también tienen el operador módulo: %, que da como resultado el resíduo de la división. Java permite este operador entre valores reales.

En el Código 4 se pueden ver ejemplos de operaciones aritméticas comunes entre enteros. Se deja como ejercicio la argumentación de por qué las líneas comentadas producen un error de compilación (el programa no compila correctamente).

```
public class IntExample{


    public static void main( String args[]){
        double doubleA= 4;
        double doubleB = 3;

        int intA = 4;
        int intB = 3;

        //int intC = 4/3.0;
        //int intD = 4.0/3;
        //int intE = 4.0/3.0;
        //int intF = doubleA/doubleB;
        //int intG = doubleA/(int)doubleB;
        //int intH = (int)doubleA/doubleB

        int intI = (int)doubleA/(int)doubleB;
        int intJ = 4/5 + 3*intB/intA - intA/intB + (intA*(intB/intA)); int
        intQuotient = 13/3;
        int intRemainder = 13%3;
        System.out.println("intA = "+intA);
        System.out.println("intB = "+intB);
        System.out.println("intQuotient = "+intQuotient);
        System.out.println("intRemainder = "+intRemainder);
        System.out.println("intI = "+intI);
        System.out.println("intJ = "+intJ);
    }
}
```

Código 4: ejemplos de operaciones entre números enteros

4.7. Operaciones booleanas

Adicional a las operaciones con tipos de datos primitivos que pueden dar como resultado números (reales o enteros), otro tipo de operaciones y expresiones en Java arrojan uno de dos posibles valores: falso o verdadero. Estas expresiones se conocen como booleanas y sirven para tomar decisiones en los programas que se escriban o para controlar la cantidad de repeticiones que se hagan de un fragmento de código. Las operaciones booleanas obedecen a los mismos principios que las operaciones del cálculo proposicional.

Al igual que las expresiones aritméticas, las booleanas pueden ser elaboradas y compuestas de más expresiones, según se requiera, para solucionar un problema determinado. Las expresiones pueden tener paréntesis anidados, tanto como sea necesario, y en caso de que no tenga, se ejecutan las operaciones según el orden de prioridad dado en Java.

En el Código 5 se muestran algunas posibilidades de operaciones y uso de operadores booleanos en Java, para ello, es necesario hacer un seguimiento paso a paso de este ejemplo y a realizar algunos cambios y experimentos removiendo/agregando paréntesis, cambiando valores y analizando las consecuencias de estas modificaciones.

```
public class BooleanExample{
    public static void main( String args[]){
        boolean and1 = true && true;
        boolean and2 = true && false;
        boolean and3 = false && true;
        boolean and4 = false && false;
        System.out.println(and1+" "+and2+" "+and3+" "+and4);
        boolean or1 = true | true;
        boolean or2 = true || false;
        boolean or3 = false || true;
        boolean or4 = false || false;
        System.out.println(or1+" "+or2+" "+or3+" "+or4);
        boolean xor1 = true ^ true;
        boolean xor2 = true ^ false;
        boolean xor3 = false ^ true;
        boolean xor4 = false ^ false;
        System.out.println(xor1+" "+xor2+" "+xor3+" "+xor4);
        boolean neg1 = !true;
        boolean neg2 = !false;
        System.out.println(neg1+" "+neg2);
        boolean b1 = ((4>2) && (3<10)) || (false && (3*4==12));
        System.out.println(b1);
        boolean b2 = ((4>2) && (3<10)) && (false && (3*4==12));
        System.out.println(b2);
        boolean b3 = ((4>2 || 1!=1) && (3<10)) && (!false && (3*4!=12));
        System.out.println(b3);
        boolean b4 = ((4>2 || 1!=1) && (3<10)) || (!false && (3*4!=12));
        System.out.println(b4);
        int ternary = 3 > 4 ? 1 : -1;
        System.out.println(ternary);
        String ternary2 = 4 == 1+4-1 ? "equal":"different";
        System.out.println(ternary2);
    }
}
```

Código 5: ejemplos de operaciones booleanas

4.7.1. Operaciones a nivel de *bits*

Los enteros, en la mayoría de lenguajes de programación de alto nivel, tienen la posibilidad de operarse a nivel de bits. Para propósitos generales, se muestran ejemplos en enteros de 32 *bits* (*int* en Java) en el código del ejemplo 6, aunque las operaciones también están definidas normalmente para enteros de más o menos precisión en los lenguajes de programación. Para profundizar acerca de los operadores a nivel de bits en cada lenguaje de programación, se puede revisar la documentación adecuada (Oracle, 1995c). De nuevo, en especial en este programa, se recomienda realizar un análisis y un seguimiento detallado de las operaciones realizadas para entender qué pasa con cada una de las variables enteras involucradas en las operaciones.

```java
public class IntsAndBitsExample{
    public static void main( String args[]){
        int a = 23, b = 27;
        System.out.println("a = "+a+" = "+Integer.toBinaryString(a)+" in binary");
        System.out.println("b = "+b+" = "+Integer.toBinaryString(b)+" in binary");
        int bitNegationA = ~a;
        int bitNegationB = ~b;
        int bitOr = a|b;
        int bitAnd = a&b;
        int bitXor = a^b;
        System.out.println("bitNegationA in Decimal: "+bitNegationA);
        System.out.println("bitNegationA in Binary : "+Integer.toBinaryString(bitNegationA));
        System.out.println("bitNegationB in Decimal: "+bitNegationB);
        System.out.println("bitNegationB in Binary : "+Integer.toBinaryString(bitNegationB));
        System.out.println(a+"|"+b+" in Decimal: "+bitOr);
        System.out.println(a+"|"+b+" in Binary : "+Integer.toBinaryString(bitOr));
        System.out.println(a+"&"+b+" in Decimal: "+bitAnd);
        System.out.println(a+"&"+b+" in Binary : "+Integer.toBinaryString(bitAnd));
        System.out.println(a+"^"+b+" in Decimal: "+bitXor);
        System.out.println(a+"^"+b+" in Binary : "+Integer.toBinaryString(bitXor));
        int shiftRight = a >> 3;
        int shiftLeft = a << 3;
        int shiftRight2 = a >>> 3;
        int shiftRightNegative = (-a) >> 3;
        int shiftLeftNegative = (-a) << 3;
        int shiftRightNegative2 = (-a) >>> 3;
        System.out.println(a+" >> "+3+" in Decimal: "+shiftRight);
        System.out.println(a+" >> "+3+" in Binary : "+Integer.toBinaryString(shiftRight));
        System.out.println(a+" << "+3+" in Decimal: "+shiftLeft);
        System.out.println(a+" << "+3+" in Binary : "+Integer.toBinaryString(shiftLeft));
        System.out.println(a+" >>> "+3+" in Decimal: "+shiftRight2);
        System.out.println(a+" >>> "+3+" in Binary : "+Integer.toBinaryString(shiftRight2));
        System.out.println(-a+" >> "+3+" in Decimal: "+shiftRightNegative);
        System.out.println(-a+" >> "+3+" in Binary : "+Integer.toBinaryString(shiftRightNegative));
        System.out.println(-a+" << "+3+" in Decimal: "+shiftLeftNegative);
        System.out.println(-a+" << "+3+" in Binary : "+Integer.toBinaryString(shiftLeftNegative));
        System.out.println(-a+" >>> "+3+" in Decimal: "+shiftRightNegative2);
        System.out.println(-a+" >>> "+3+" in Binary : "+Integer.toBinaryString(shiftRightNegative2));
    }
}
```

Código 6: ejemplos de operaciones a nivel de bits

4.7.2. Conjuntos y operaciones a nivel de bits

Una de las maneras más comunes de aprovechar las operaciones a nivel de bits en los lenguajes de programación de alto nivel es usarlos para encapsular un conjunto de elementos en un solo entero. Esto se hace asumiendo un conjunto de máximo 31 elementos (para un entero de 32 bits). Si en la representación binaria de la variable entera en cuestión hay un 1, significa que el elemento representado por esa posición está en el conjunto; si hay un 0, se asume que no lo está. El uso de enteros como abstracciones de conjuntos puede ser utilizado para realizar cálculos de propiedades o de selecciones óptimas de candidatos a gran velocidad, lo cual ahorra un espacio considerable en la memoria. En la actualidad, este tema ha cobrado mayor importancia con el internet de las cosas . También, puede servir para hacer de manera eficiente el envío de datos a través de redes. En los códigos 7 y 8 se muestra el funcionamiento de las operaciones más conocidas de conjuntos.

En esos ejemplos se asume un conjunto de 4 elementos, a saber: A = {a, b, c, d}. En un entero que represente un conjunto con estos elementos, el *bit* menos significativo, el bit en la posición 0 estará prendido (en 1) si el elemento a está en el conjunto; el bit en la posición 1 estará prendido si el elemento b está en el conjunto; el bit en la posición 2 estará prendido si el elemento c está en el conjunto; el *bit* en la posición 3 estará prendido si el elemento ***d*** está en el conjunto. Para mayor claridad, en la siguiente lista se muestran todas las posibilidades de subconjuntos del conjunto A y sus variaciones a través de la representación binaria de un entero. Hay que anotar que los índices se dan de derecha a izquierda, siendo el índice 0 el bit que está más a la derecha, es decir, el menos significativo, el que representa la a en el conjunto; el segundo de derecha a izquierda representa la ***b*** y así sucesivamente.

- 0 en decimal, 0 en binario representa el conjunto {}
- 1 en decimal, 1 en binario representa el conjunto {a}
- 2 en decimal, 10 en binario representa el conjunto {b}
- 3 en decimal, 11 en binario representa el conjunto {a,b}
- 4 en decimal, 100 en binario representa el conjunto {c}
- 5 en decimal, 101 en binario representa el conjunto {a,c}
- 6 en decimal, 110 en binario representa el conjunto {b,c}
- 7 en decimal, 111 en binario representa el conjunto {a,b,c}
- 8 en decimal, 1000 en binario representa el conjunto {d}
- 9 en decimal, 1001 en binario representa el conjunto {a,d}
- 10 en decimal, 1010 en binario representa el conjunto {b,d}
- 11 en decimal, 1011 en binario representa el conjunto {a,b,d}
- 12 en decimal, 1100 en binario representa el conjunto {c,d}
- 13 en decimal, 1101 en binario representa el conjunto {a,c,d}
- 14 en decimal, 1110 en binario representa el conjunto {b,c,d}
- 15 en decimal, 1111 en binario representa el conjunto {a,b,c,d}

```
import static java.lang.Integer.*; public
class IntsAndBitsExample{
    public static void main( String args[])    {
        //To represent the set: {a,b,c,d}
        String universalSetNames = "dcba";
        int universalSetSize = universalSetNames.length(); int
        universalSet = (1<<universalSetSize)-1;
        System.out.println("{d,c,b,a} = "+universalSet+" =  "+
                toBinaryString(universalSet));

        int indexA = 0;
        int indexB = 1;
        int indexC = 2;
        int indexD = 3;

        int aBit = 1<<indexA; int
        bBit = 1<<indexB; int
        cBit = 1<<indexC; int
        dBit = 1<<indexD;

        System.out.println("a = "+aBit+" = "+toBinaryString(aBit)+" in binary");
        System.out.println("b = "+bBit+" = "+toBinaryString(bBit)+" in binary");
        System.out.println("c = "+cBit+" = "+toBinaryString(cBit)+" in binary");
        System.out.println("d = "+dBit+" = "+toBinaryString(dBit)+" in binary");

        int aUc = aBit|cBit;
        System.out.println("aUc = "+aUc+" = "+toBinaryString(aUc)+" in binary");
        int dUc = dBit|cBit;
        System.out.println("dUc = "+dUc+" = "+toBinaryString(dUc)+" in binary");
        int intersect = aUc & dUc;
        System.out.println("{c,a}Intersect{d,c} = "+intersect+" = "+
                toBinaryString(intersect)+" in binary");
        int difference = universalSet & ~ aUc;
        System.out.println("{d,c,b,a}-{c,a} = "+difference+" = "+
                toBinaryString(difference)+" in binary");
```

Código 7: conjuntos y operaciones de bits 1/2

```
        int simetricDiff= aUc ^ dUc;
        System.out.println("{a,c}Simetric Difference{d,c} = "+simetricDiff+" = "+
                toBinaryString(simetricDiff)+" in binary");

        int allExceptC =  universalSet & (~(1 << indexC));
        System.out.println("{d,c,b,a} Except {c} = "+allExceptC+" = "+
                toBinaryString(allExceptC)+" in binary");

        boolean isAin_aUc = (aUc & 1 << indexA) != 0;
        boolean isBin_aUc = (aUc & 1 << indexB) != 0;
        boolean isCin_aUc = (aUc & 1 << indexC) != 0;
        boolean isDin_aUc = (aUc & 1 << indexD) != 0;

        System.out.println("a in {a,c} = "+isAin_aUc);
        System.out.println("b in {a,c} = "+isBin_aUc);
        System.out.println("c in {a,c} = "+isCin_aUc);
        System.out.println("d in {a,c} = "+isDin_aUc);

        int testSet = 0;//The empty set
        System.out.println("before adding 'a' = "+testSet+" = "+
                toBinaryString(testSet)+" in binary");
        testSet |= 1 << indexA;//add 'a' to the set
        System.out.println("{} after adding 'a' = "+testSet+" = "+
                toBinaryString(testSet)+" in binary");
        testSet |= 1 << indexB;//add 'b' to the set
        System.out.println("{a} after adding 'b' = "+testSet+" = "+
                toBinaryString(testSet)+" in binary");
        testSet |= 1 << indexC;//add 'c' to the set
        System.out.println("{b,a} after adding 'c' = "+testSet+" = "+
                toBinaryString(testSet)+" in binary");
        testSet |= 1 << indexD;//add 'd' to the set
        System.out.println("{c,b,a} after adding 'd' = "+testSet+" = "+
                toBinaryString(testSet)+" in binary");

        int sizeOfASet = bitCount(aUc);
        System.out.println("size of "+aUc+" ("+toBinaryString(aUc)+") = "+sizeOfASet);
    }
}
```

Código 8: conjuntos y operaciones de bits 2/2

En los códigos 7 y 8 se importó de manera estática la clase *Integer*, que es la clase envolvente para el tipo de dato usado (*int*). Se hace así cuando en un programa se quiere usar los métodos de esa clase (funciones o procedimientos en otros lenguajes) repetidamente y no hay lugar a la ambigüedad (varios métodos en diferentes clases se llaman igual). En particular, se usa bastante el método ***toBinaryString***, el cual transforma un entero en su representación binaria para facilitar el entendimiento del ejemplo. También se usa el método ***bitCount***, el cual retorna la cantidad de unos en la representación binaria del número que recibe; este número que recibe el método está entre paréntesis, y en el ejemplo dado representa un conjunto.

Como se puede ver en este ejemplo, el nombramiento adecuado de variables contribuye a la legibilidad del código hecho. En el Código 9 se ven otras operaciones estándar en Java para manipular bits. Aunque no son tan comunes en otros lenguajes de programación, pueden ser útiles en ciertos contextos; para conocer en mayor detalle estas operaciones, se debe consultar la documentación de Java en la clase *Integer* (Oracle, 1995f).

```
import static java.lang.Integer.*;
public class ExtraBitsAndSetsExample{
    public static void main( String args[])     {
        //To represent the set: {a,b,c,d}
        String universalSetNames = "dcba";
        int universalSetSize = universalSetNames.length();
        int universalSet = (1<<universalSetSize)-1;
        System.out.println("{d,c,b,a} = "+universalSet+" =  "+
                toBinaryString(universalSet));
        int indexA = 0;
        int indexC = 2;
        int aBit = 1<<indexA;
        int cBit = 1<<indexC;

        int aUc = aBit|cBit;
        System.out.println("aUc = "+aUc+" = "+toBinaryString(aUc)+" in binary");
        int sizeOfASet = bitCount(aUc);
        System.out.println("size of "+aUc+" ("+toBinaryString(aUc)+") = "+sizeOfASet);

        System.out.println(highestOneBit(aUc));
        System.out.println(lowestOneBit(aUc));
        System.out.println(Integer.numberOfLeadingZeros(aUc));
        System.out.println(Integer.numberOfTrailingZeros(aUc));
        System.out.println(toBinaryString(Integer.reverse(aUc)));
        System.out.println(toBinaryString(reverseBytes(aUc)));
        System.out.println(toBinaryString(rotateLeft(aUc, 2)));
        System.out.println(toBinaryString(rotateRight(aUc, 2)));
        System.out.println(Integer.toHexString(aUc));
        System.out.println(Integer.toOctalString(aUc));
    }
}
```

Código 9: Otras operaciones a nivel de *bits* en Java

4.8. Enteros de precisión arbitraria

En ocasiones, durante el desarrollo de programas, es necesario trabajar con enteros que sobrepasan los valores que se pueden almacenar en los tipos de datos nativos. En Java este límite está dado por los valores máximos del tipo de dato *long*. Algunos lenguajes, como Java, ofrecen un tipo de dato que sirve para almacenar enteros arbitrariamente grandes y ofrece algunas operaciones elementales (y otras no tan elementales) para trabajar con ellos. En el Código 10 se muestran algunas de las operaciones y constantes disponibles para el tipo de dato *BigInteger* en Java.

Para más detalles y conocer otras operaciones sofisticadas, no solo aritméticas, sino con aplicaciones en teoría de números o a nivel de bits, se recomienda ver Oracle(1995b).

```
import java.math.BigInteger;
public class BigIntegerExample{
    public static void main( String args[]) {
        System.out.println(BigInteger.ONE);
        System.out.println(BigInteger.ZERO);
        System.out.println(BigInteger.TEN);

        BigInteger a = new BigInteger("600000000000000000000000000000000000000000");
        BigInteger b = new BigInteger("200000000000000000000000000000000000001234");
        BigInteger c = a.add(b);
        System.out.println(c);
        c = a.multiply(b);
        System.out.println(c);
        c = a.subtract(b);
        System.out.println(c);
        c = a.divide(b);
        System.out.println(c);

    }
}
```

Código 10: *BigInteger* en Java

4.9. Números reales de precisión arbitraria

Al igual que con los enteros, también hay herramientas en algunos lenguajes para trabajar con números reales de precisión mayor a la que dan los tipos de datos nativos como *double* y *float*. En Java esta herramienta se llama *BigDecimal*, y tiene métodos (operaciones) y constantes similares a las que de los *BigIntegers*. El uso de *BigDecimal* en Java se debe hacer con precaución, ya que si no se establece una precisión límite para las operaciones, se puede incurrir en la generación de un error en tiempo de ejecución en el caso de que el resultado de una operación sea decimales infinitos. Para prevenir este error es recomendable el uso de la clase *MathContext* con el objetivo de definir una precisión arbitraria, que puede ser bastante más grande que la dada por los tipos nativos *double* y *float*. En el Código 11 se puede ver un ejemplo del uso de estas dos clases en algunas operaciones elementales. Se recomienda revisar Oracle (1995a) para conocer más detalles sobre el uso, funcionamiento y posibilidades de la clase *BigDecimal* con el propósito de manejar números reales de precisión arbitraria.

```
import java.math.BigDecimal; import
java.math.MathContext; public class
BigDecimalExample{
    public static void main( String args[]){
        System.out.println(BigDecimal.ONE);
        System.out.println(BigDecimal.ZERO);
        System.out.println(BigDecimal.TEN);

        BigDecimal a = new BigDecimal("600000000000000000000000000000000.00000929309");
        BigDecimal b = new BigDecimal("200000000000000000000000000000000.00034211234");
        System.out.println("Precision: "+a.precision());
        MathContext mc = new MathContext(100);
        BigDecimal c = a.add(b,mc);
        System.out.println(c);
        c = a.multiply(b);
        System.out.println(c);
        c = a.subtract(b);
        System.out.println(c);
        c = a.divide(b,mc);
        System.out.println(c);

    }
}
```

Código 11: *BigDecimal* en Java

4.10. Operaciones con caracteres y cadenas de caracteres

Tal vez el tipo de dato no nativo de más uso en lenguajes de programación es el que sirve para almacenar texto de longitud arbitraria. En el caso de Java hay varias opciones, entre ellas la clase *String* (otras son *StringBuilder* y *StringBuffer*). Esta clase encapsula el comportamiento de una cadena de caracteres y ofrece una buena cantidad de comportamientos (métodos u operaciones) que facilitan su uso y el diseño e implementación de algunos algoritmos. En el Código 12 se pueden ver algunas operaciones comunes.

```
public class StringAndCharExamples{
    public static void main( String args[])    {
        String s = "  Strings in Java are easy  "; String
        s2 = "  Strings in Java ARE EASY  ";
        System.out.println(s.charAt(4));
        System.out.println(s.charAt(3));
        System.out.println(s.charAt(2));
        System.out.println(s.charAt(1));
        System.out.println(s.charAt(0));
        System.out.println("The size is :"+s.length());
        System.out.println(s.equalsIgnoreCase(s2));
        System.out.println(s.equals(s2));
        System.out.println(s.compareTo("XXX"));
        System.out.println(s.compareTo("AAA"));
        System.out.println(s.compareTo(s));
        System.out.println(s.compareToIgnoreCase(s2));
        System.out.println(s.toUpperCase());
        System.out.println(s.toLowerCase());
        System.out.println(s.indexOf("Java"));
        System.out.println(s.indexOf("xxxx"));
        System.out.println(s);
        System.out.println("_"+s.trim()+"_");
        System.out.println(s.trim().endsWith("easy"));
        System.out.println(s.trim().endsWith("hard"));
        System.out.println(s.trim().startsWith("Strings"));
        System.out.println(s.trim().startsWith("games"));
        System.out.println(s.lastIndexOf("s"));
    }
}
```

Código 12: operaciones comunes de cadenas en java

4.10.1. Usos avanzados de cadenas de caracteres

Adicional a las operaciones estándar de cadenas de caracteres que se tienen en lenguajes de programación de alto nivel, hay muchas otras con diferentes aplicaciones, que no están dentro del alcance de este módulo. Algunas de ellas involucran la creación y uso de estructuras de datos avanzados, como *Suffix Trees* y *Suffix Arrays*, para la búsqueda de patrones en textos largos. Otras involucran estructuras auxiliares o técnicas avanzadas de programación para ciertos tipos de búsquedas de patrones. En la literatura hay bastante material sobre el uso eficiente de cadenas de texto y la búsqueda de patrones en ellas. Un buen punto de partida es Charras y Lecroq (2004).

4.11. Transformaciones entre tipos de datos

En prácticamente todos los lenguajes de programación conviene en algunas ocasiones tratar ciertos datos como texto para poderlos trabajar, y en otras oportunidades es más adecuado considerarlos como un tipo de dato nativo para operar con ellos.

Es por esto que que los lenguajes tienen diferentes maneras de transformar entre sí tipos de datos nativos y tipos de datos cadena (*String*). En Java toman importancia las clases envolventes, que son estándar de Java, diseñadas para encapsular el comportamiento de un dato nativo. Estas son: Byte, *Short*, *Integer*, *Long*, *Float*, *Double*, *Boolean* y *Character*, las cuales encapsulan, respectivamente, los tipos de datos nativos byte, *short*, *int*, *long*, *float*, *double* y *boolean*. Las transformaciones de tipos de datos nativos a cadenas se pueden hacer de varias formas (ver los códigos 13 y 14).

```
public class TypeConversions{
    public static void main( String args[])        {
        byte b = 1;
        short s = 2;
        int     i = 3;
        long l = 4;
        float f = 5;
        double d = 6;
        char c = 'x';
        boolean     bn = true;

        String strB = b+"";
        String strS = String.valueOf(s);
        String strI = new String(i+"");
        String strL = l+"";
        String strF = new String( String.valueOf(f)); String strD =
        new String( ""+d);
        String strC = c+"";
        String strBn = bn+"";
        System.out.println("Printing Strings Values.");
        System.out.println(strB+" "+strS+" "+strI+" "+strL);
        System.out.println(strF+" "+strD+" "+strS+" "+strC);
        System.out.println(strBn);

        b = Byte.parseByte( strB );
        s = Short.parseShort( strS );
        i = Integer.parseInt( strI );
        l = Long.parseLong( strL );
        f = Float.parseFloat( strF );
        d = Double.parseDouble( strD );
        // not parse, we must extract first character
        c = strC.charAt(0);
        bn = Boolean.parseBoolean(strBn);
        System.out.println("Printing native variables via parsing.");
        System.out.println(b+" "+s+" "+i+" "+l);
        System.out.println(f+" "+d+" "+s+" "+c);
        System.out.println(bn);
```

Código 13: operaciones comunes de cadenas en Java

```
        b = Byte.valueOf( strB );
        s = Short.valueOf( strS ); i =
        Integer.valueOf( strI );
        l = Long.valueOf( strL );
        f = Float.valueOf( strF );
        d = Double.valueOf( strD );
        bn = Boolean.valueOf(strBn);
        System.out.println("Printing native variables via valueOf.");
        System.out.println(b+" "+s+" "+i+" "+l);
        System.out.println(f+" "+d+" "+s+" "+c);
        System.out.println(bn);

        Byte bw = 1;
        Short sw = 2;
        Integer iw = 3;
        Long lw = 4L;
        Float fw = new Float(5);
        Double dw = new Double(6);
        Character cw = 'x';
        Boolean     bnw = true;
        System.out.println("Printing wrapper values.");
        System.out.println(bw+" "+sw+" "+iw+" "+lw);
        System.out.println(fw+" "+dw+" "+sw+" "+cw);
        System.out.println(bnw);

        b = bw.byteValue();
        s = sw.shortValue();
        i = iw.intValue();
        l = lw.longValue();
        f = fw.floatValue();
        d = dw.doubleValue();
        bn = bnw.booleanValue();
        System.out.println("Printing native values methods in wrapper classes.");
        System.out.println(b+" "+s+" "+i+" "+l);
        System.out.println(f+" "+d+" "+s+" "+c);
        System.out.println(bn);
    }
}
```

Código 14: Operaciones comunes de cadenas en Java

5. Problemas propuestos

Todos los problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java, que tome como entrada ciertos datos leídos desde su entrada estándar y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En los problemas propuestos se deben asumir que las entradas cumplen con lo estipulado en la sección "**`Descripción de entrada`**" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo definido en la sección "**`Descripción de salida`**" y que sea únicamente lo solicitado allí.

1. Par o impar.
   Diseñar un programa que tome como entrada un número entero y muestre en la salida *even* si el número es par, y *odd* si es impar. No usar condicionales.
   - Descripción de entrada: un número entero.
   - Descripción de salida: *odd* o *even*, según sea el caso.
   - Ejemplo de entrada:
     126
   - Ejemplo de salida:
     even
2. Múltiplo de 100.
   Diseñar un programa que tome como entrada un número entero y diga si es múltiplo de 100 o no.
   - Descripción de entrada: Un número entero.
   - Descripción de salida: multiple of 100 o not multiple of 100, según sea el caso.
   - Ejemplo de entrada:
     12600
   - Ejemplo de salida:
     multiple of 100
3. Evaluador de igualdad de reales.
   En muchas ocasiones se deben comparar números reales (*double* o *float)* para determinar igualdad. La manera correcta de hacerlo es establecer (o recibir) una tolerancia, que es generalmente un valor muy pequeño de diferencia permitida. Con base en esa tolerancia, hay que decidir si los números son iguales o no. Esto porque, en algunos ámbitos, los números 3 y 2.999999999 deben ser considerados iguales; por ejemplo, si son magnitudes en pesos y se trata del sistema contable de un banco. Pero si los mismos dos números son magnitudes en metros, de valores de un sistema de información de elementos de magnitudes nanoscópicas, seguramente no deban considerarse iguales. En este problema se debe hacer un programa que reciba 3 números; los dos primeros son los números reales a comparar y el último es la tolerancia dada. Los números se consideran iguales si su diferencia es máximo la tolerancia dada, pero no la sobrepasa.
   - Descripción de entrada: tres números reales; el último de ellos será positivo con toda seguridad.
   - Descripción de salida: *equals* si los dos primeros números son iguales según la tolerancia dada; *not equals* si no lo son.
   - Ejemplo de entrada:
     2.999 3 0.00000001
   - Ejemplo de salida:
     equals

- Ejemplo de entrada 2:
  3 2.999 0.00000001
- Ejemplo de salida 2:
  equals
- Ejemplo de entrada 3:
  2.999 3 0.01
- Ejemplo de salida 3:
  not equals
- Ejemplo de entrada 4:
  3 2.999 0.01
- Ejemplo de salida 4:
  not equals

4. Programa extractor de año a partir de la tarjeta de identidad.
   El programa debe producir un número entero indicando el año que está en el número leído, asumiendo que tiene el formato de una tarjeta de identidad.
   - Descripción de entrada: un número entero positivo de exactamente 11 dígitos, en el cual el dígito más significativo (más hacia la izquierda) no es 0.
   - Descripción de salida: un número entero según lo solicitado en el enunciado.
   - Ejemplo de entrada:
     86021912345
   - Ejemplo de salida:
     86
5. Programa que prueba si un valor es raíz de un polinomio de grado dos.
   - Descripción de entrada: 4 valores enteros; los primeros 3 serán los coeficientes del polinomio ax2+bx+c = 0, en el orden en que aparecen. El cuarto valor será el número x (otro entero) para validar si es o no raíz. Si efectivamente una lo es, debe mostrar en la salida estándar *root!*; de lo contrario, sale *not root!*. El programa no puede usar condicionales (*if*) ni ciclos, ni funciones. Recomendación: usar el *if* ternario (?:).
   - Descripción de salida: *root* o *not root*! según el caso.
   - Ejemplo de entrada:
     1 -5 6 3
   - Ejemplo de salida:
     root!
   - Ejemplo de entrada 2:
     1 -5 6 2
   - Ejemplo de salida 2:
     root!
   - Ejemplo de entrada 3:
     1 -5 6 7
   - Ejemplo de salida 3:
     not
6. Solucionar una ecuación cuadrática.
   - Descripción de entrada: 3 valores enteros, que serán los coeficientes del polinomio ax2 + bx + c = 0, en el orden en que aparecen.
   - Descripción de salida: la salida deben ser dos números reales, las dos soluciones a la ecuación cuadrática (Wikipedia, 2018). Se debe asumir que los valores existen. Usar el método en la clase *Math* para cálculo de raíces: *Math.sqrt*(). En la salida debe estar primero la menor raíz. Nota: no usar condicionales (*if*).

- Ejemplo de entrada:
  1 -5 6
- Ejemplo de salida:
  2 3

7. Modificar el programa anterior para que detecte cuándo un polinomio no tiene soluciones reales y muestre un mensaje correspondiente. Nota: no usar condicionales; usar *if* ternario y transformaciones entre tipos de datos nativos y cadenas para la salida del programa.
   - Descripción de entrada: 3 valores enteros, los cuales serán los coeficientes del polinomio ax2 + bx + c = 0, en el orden en que aparecen.
   - Descripción de salida: las dos soluciones, igual que el problema anterior, en caso de que existan, o un mensaje que diga que no existen.
   - Ejemplo de entrada:
     -11 -10
   - Ejemplo de salida:
     There is no solution in the real numbers.

8. Calculador de mayor de 5 números. No usar condicionales ni métodos, ni funciones estándar, como Math.min() o Math.max().
   - Descripción de entrada: 5 números enteros.
   - Descripción de salida: el mayor de los 5 números.
   - Ejemplo de entrada:
     6 9 8 7 5
   - Ejemplo de salida:
     9
9. Calculador del menor de 5 números. No usar condicionales, ni métodos ni funciones estándar, como Math.min() o Math.max().
   - Descripción de entrada: 5 números enteros.
   - Descripción de salida: el mayor de los 5 números.
   - Ejemplo de entrada:
     6 9 8 7 5
   - Ejemplo de salida:
     5
10. Cantidad de baldosas. Suponiendo que se tienen baldosas cuadradas de X por X metros y se desea cubrir totalmente una superficie rectangular de dimensiones N por M metros sin partir ni poner ninguna baldosa sobre otra (aunque pueda quedar sobrando baldosa), ¿cuál será el menor número de baldosas que se deben usar? Ejemplo: la superficie a cubrir es de 16 por 23 metros con baldosas cuadradas de 3 metros de lado. La solución óptima es usar 48 baldosas; esto hará que se pueda cubrir toda la superficie y quedarán algunas baldosas parcialmente por fuera de esta. La tarea es hacer un programa que lea estos tres valores X, N y M, en ese orden, y como salida muestre un solo número indicando la cantidad mínima de baldosas cuadradas de lado X para cubrir un área rectangular de tamaño N × M. No usar condicionales, ni ciclos ni métodos ni funciones estándar del lenguaje en la clase Math.
    - Descripción de entrada: tres valores enteros positivos en el orden dado en el enunciado: X N M .
    - Descripción de salida: un valor entero según lo descrito en el enunciado.
    - Ejemplo de entrada:
      3 16 23
    - Ejemplo de salida:
      48

- Ejemplo de entrada 2:
  5 4 3
- Ejemplo de salida 2:
  1
- Ejemplo de entrada 3:
  5 10 30
- Ejemplo de salida 3:
  6
- Ejemplo de entrada 4:
  5 11 30
- Ejemplo de salida 4:
  9
- Ejemplo de entrada 5:
  5 10 31
- Ejemplo de salida 5:
  8

Referencias

Blair, I. (2017). *14 Programming Languages for Mobile App Development*. Recuperado de https://buildfire.com/programming-languages-for-mobile-app-development/

Charras, C. & Lecroq, T. (2004). *Handbook of Exact String Matching Algorithms*. London, U.K.: King´s College Publications.

Deitel, H. & Deitel, P. (1999). *Java - How to Program*. Prentice Hall.

Gabbrielli, M. & Martini, S. (2010). *Programming Languages: Principles and Paradigms (Undergraduate Topics in Computer Science)*. New York, US:ASpringer Science & Business Media.

Louden, K. & Lambert, K. (2011). *Programming languages: principles and practices*. Cengage Learning.

Oracle. (1995a). *Class BigDecimal*. Recuperado de https://docs.oracle.com/Javase/7/docs/api/Java/math/BigDecimal.html

Oracle. (1995b). *Class BigInteger*. Recuperado de https://docs.oracle.com/Javase/7/docs/api/Java/math/BigInteger.html

Oracle. (1995c). *Bitwise and Bit Shift Operators*. Recuperado de https://docs.oracle.com/Javase/tutorial/Java/nutsandbolts/op3.html

Oracle. (1995d). *Chapter 4. Types, Values, and Variables*. Recuperado de https://docs.oracle.com/Javase/specs/jls/se7/html/jls-4.html

Oracle. (1995e). *Chapter 5. Conversions and promotions in Java*. Recuperado de https://docs.oracle.com/javase/specs/jls/se7/html/jls-5.html

Oracle. (1995f). *Class Integer*. Recuperado de https://docs.oracle.com/Javase/7/docs/api/Java/lang/Integer.html

Oracle. (1995g). *Java Language Keywords*. Recuperado de https://docs.oracle.com/Javase/tutorial/Java/nutsandbolts/ keywords.html

Oracle. (1999a). *Comments*. Recuperado de http://www.oracle.com/technetwork/Java/Javase/documentation/codeconventions-141999.html

Oracle. (1999b). *Primitive Data Types*. Recuperado de https://docs. oracle.com/Javase/tutorial/Java/nutsandbolts/datatypes.html

Oracle. (1999c). *Class Gregorian Calendar*. Recuperado de https://docs.oracle.com/Javase/7/docs/api/Java/util/GregorianCalendar.html

Ovid. (19 de agosto de 2010). *What to know before debating type systems* [Mensaje de un blog]. Recuperado de http://blogs.perl.org/users/ovid/2010/08/what-to-know-before-debating-type-systems.html

Putano, B. (18 de diciembre de 2017). *Most Popular and Influential Programming Languages of 2018* [Mensaje de un blog]. Recuperado de https://stackify.com/popular-programming-languages-2018/

Quora. (2018). *Quora webpage*. Recuperado de https://www.quora.com/

Sebesta, R. W. (2012). *Concepts of Programming Languages*. Harlow, U.K: Pearson.

Sierra, K y Bates, B. (2005). *Head First Java*. Sebastopol, CA, USA: O´Reilly.

StackExchange. (2018). *StackExchange*. Recuperado de https://stackexchange.com/

StackOverflow.(2018). *StackOverflow webpage*. Recuperado de https://stackoverflow. com/

TopCoder (2018). *TopCoder Forums*. Recuperado de https://apps.topcoder.com/forums/

Van Roy, P. y Haridi, S. (2004). *Concepts, Techniques, and Models of Computer Programming*. Cambridge, MA, USA: The MIT Press.

Watt, D. (2004). *Programming language design concepts*. Chichester, U.K.: John Wiley & Sons.

Wikipedia. (2018). *Quadratic Formula*. Recuperado de https://en.wikipedia.org/wiki/ Quadratic formula

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 1: Introducción a la programación
Escenario 1: Conceptos iniciales en lenguajes de programación

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.